

Project Proposal: Personal Weekly Spotify Wrapped

High level description of the task that you want to automate

I want to automate the weekly analysis and documentation of my personal music listening habits. Currently, Spotify only provides an annual "Wrapped" overview, and manual tracking is highly inefficient. This Python project will automatically retrieve my recently played tracks on a weekly basis, store the data locally to overcome the API's track limit, and generate a personalized weekly summary report.

High level steps that are needed to automate

- **Authentication:** Securely authenticate with the Spotify Web API using OAuth and local environment variables.
- **Data Retrieval:** Fetch the "Recently Played" tracks data (including track names, artists, duration, and timestamps) from the API.
- **Data Storage:** Process and insert the retrieved data into a local SQLite database to prevent duplicates and build a continuous listening history.
- **Data Aggregation:** Calculate weekly statistics (e.g., total minutes listened, top artists, top tracks) by querying the local database.
- **Report Generation:** Generate a formatted weekly report (e.g., as a PDF, CSV, or text file) and save it to the local filesystem.

Systems that will be involved (Database, Filesystem, APIs)

- **APIs:** Interaction with the official Spotify Web API (REST API) to retrieve user data.
- **Database:** A local SQLite database to permanently store the listening history, as the API only returns the 50 most recent tracks.
- **Filesystem:** Writing the generated weekly analysis reports to a local directory.
- **Operating System:** Utilizing environment variables (e.g., via a .env file) to securely store sensitive API credentials like the Client ID and Client Secret.

Estimated benefits of automating (hours saved, quality of life improvement, cost savings)

- **Hours saved:** Saves approximately 1 to 2 hours per week that would otherwise be required to manually track, log, and calculate listening times and top artists.
- **Quality of life improvement:** Provides immediate, consistent, and personalized insights into my music trends without having to rely on third-party tracking apps or waiting a full year for official statistics.

Improved Requirement Additions for M122E Grading

The original project idea stays the same. The following red additions make the requirements more explicit, prioritized and measurable so that the Functional Requirements and Non-Functional Requirements criteria are covered more completely.

Clarified Automation Scope

The final script will not only retrieve Spotify data. It will use Last.fm as the main source for weekly listening history, Spotify as an enrichment API for track details, SQLite for local storage, the filesystem for HTML report output, and Gmail SMTP for automatic email delivery.

- Last.fm provides the weekly scrobble history for the last 7 days, including track, artist, album, timestamp and Last.fm URL if available.
- Spotify enriches the Last.fm tracks with Spotify links, duration, album information and cover images where possible.
- Gmail SMTP sends the generated weekly report automatically to the configured recipient email address.
- The script must continue running with Last.fm data if Spotify enrichment or the optional playlist update fails.

Functional Requirements - MoSCoW Prioritized

ID	Priority	Requirement
FR-01	Must	The script shall load all required configuration values from a .env file and validate missing values at startup.
FR-02	Must	The script shall fetch the user's Last.fm tracks from the last 7 days.
FR-03	Must	The script shall store listening history in a local SQLite database.
FR-04	Must	The database shall prevent duplicate track plays using track name, artist and timestamp.
FR-05	Must	The script shall calculate weekly statistics such as top songs, top artists, total tracks and estimated listening time.
FR-06	Must	The script shall generate a readable weekly HTML report and save it in the output folder.

FR-07	Must	The script shall send the weekly report by email using Gmail SMTP when email sending is enabled.
FR-08	Should	The script should enrich tracks with Spotify metadata such as track URL, album name, cover URL and duration.
FR-09	Should	The script should log important workflow steps and clear error messages.
FR-10	Could	The script could create or update a Spotify playlist with the top songs of the week.
FR-11	Won't	The script will not implement a graphical user interface because the project focuses on automation through a Python script.

Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security	API keys, app passwords and tokens must only be stored in .env and must never be hardcoded in Python files.
NFR-02	Reliability	The script must handle API or email errors with clear messages instead of crashing without explanation.
NFR-03	Maintainability	The code must be split into clear modules for configuration, APIs, database, reports and email sending.
NFR-04	Testability	Core functionality must be testable with pytest, especially config validation, duplicate prevention and report/statistics logic.
NFR-05	Usability	The project must include a README with setup, environment variables, execution steps and test instructions.
NFR-06	Performance	The script should finish a weekly run in a reasonable time for normal personal listening history data.

Acceptance Criteria

- Running python main.py creates or updates the SQLite database automatically.
- Running python main.py creates an HTML report in the output folder.

- With valid Gmail credentials and SEND_EMAIL=true, the report email is delivered successfully.
- Running python -m pytest executes the unit tests successfully.
- If Spotify playlist permissions fail, the script logs a warning and still completes the report and email workflow.

Improved Benefits of Automation

Besides saving approximately 1 to 2 hours per week, the automation improves data consistency, avoids manual calculation errors, keeps a local historical listening database, and makes the weekly music summary repeatable through scheduled execution.